Karachi AQI Predictor

Forecasting Air Quality with Machine Learning & MLOps

Internship Report

Prepared by: Sumeet Kumar
Institution: IBA Karachi
Host Organisation: 10 Pearls
Report Date: September 2026

1. Background and Purpose

This assignment, set by 10 Pearls, stood apart from the Kaggle-style coursework I had grown used to during my Machine Learning class at IBA Karachi. Rather than starting from a clean, ready-to-use dataset, the task called for an end-to-end system: pulling live data through external APIs, engineering features from it, training models on it, and finally putting the whole thing into production. That scope made the work considerably harder, but also far closer to how machine learning is actually practiced outside the classroom.

At its core, the system forecasts the Air Quality Index (AQI) category for Karachi, Pakistan, looking as far as 72 hours ahead. What makes AQI an interesting target is that it is shaped by two quite different kinds of drivers:

- **Pollutant concentrations** — particulate matter such as PM2.5, which feeds directly into the AQI rating.
- **Weather conditions** — temperature, humidity, wind speed, rainfall, and pressure, which govern how those pollutants spread out or build up.

Recognising that a working model needed to capture both of these dimensions at once — and combine them meaningfully — was one of the more valuable lessons the project offered beyond anything covered in lectures.

1.1 Settling on a Target Variable

One of the bigger design questions in the project was how to define what the model should actually predict. Two candidate approaches were weighed against each other:

- **Approach A — Categorical classification:** take the AQI category (1-5) already computed by the OpenWeather API and predict that directly as a classification target.
- **Approach B — PM2.5 regression:** predict the continuous PM2.5 concentration itself with a regression model, then convert that number into a standard 0-500 AQI scale using the official US EPA formula.

Approach A was the initial choice, mainly for its simplicity — building on the API's pre-computed index kept the pipeline lean and let feature engineering take priority, without extra mathematical conversion steps. It gave a fast, workable baseline early on. As the project progressed, though, the downside of that simplicity became clear: compressing a continuous pollutant signal into just five buckets throws away a lot of the underlying variance and makes it harder for a model to learn fine-grained environmental patterns. That pushed the design toward Approach B.

Approach B is more demanding — it requires regression rather than classification, the EPA conversion formula, and regression-specific metrics such as RMSE and MAE — but it pays off with more reliable, more interpretable results. Working with continuous physical measurements lets the model pick up on high-resolution atmospheric behaviour that categorical buckets would otherwise hide. Moving from Approach A to Approach B brought the system closer to how environmental forecasting is actually done in industry, and made the end result more physically grounded.

2. The Dataset

2.1 At a Glance

Property	Value
Location	Karachi, Pakistan (24.8608°N, 67.0104°E)
Time span	78 days
Sampling frequency	Hourly
Total records	1,872 hourly observations
Number of engineered features	30
Prediction target	PM2.5
Data sources	Open-Meteo (weather) and OpenWeather (AQI)

2.2 Grouping the Features

The 30 engineered features were organised into several groups, designed to capture both the present-moment conditions and how those conditions have been trending over time.

Category	Count	What it captures
Raw weather	5	Temperature, humidity, wind speed, wind direction, cloud cover
Raw pollution	8	PM2.5, PM10, CO, NO, NO2, O3, SO2, NH3 (µg/m3)
Target	1	AQI category from the OpenWeather API
Time-based	5	Hour, day, month, weekday, cyclical sin/cos encodings
Lag features	3	AQI values from 24h, 48h and 72h earlier (to avoid leakage)
Rolling statistics	6+	72-hour rolling means and standard deviations
Derived features	8+	Weather interaction terms, trend indicators

2.3 How the AQI Classes Are Distributed

The prediction target falls into four categories. Their spread across the collected records is shown below.

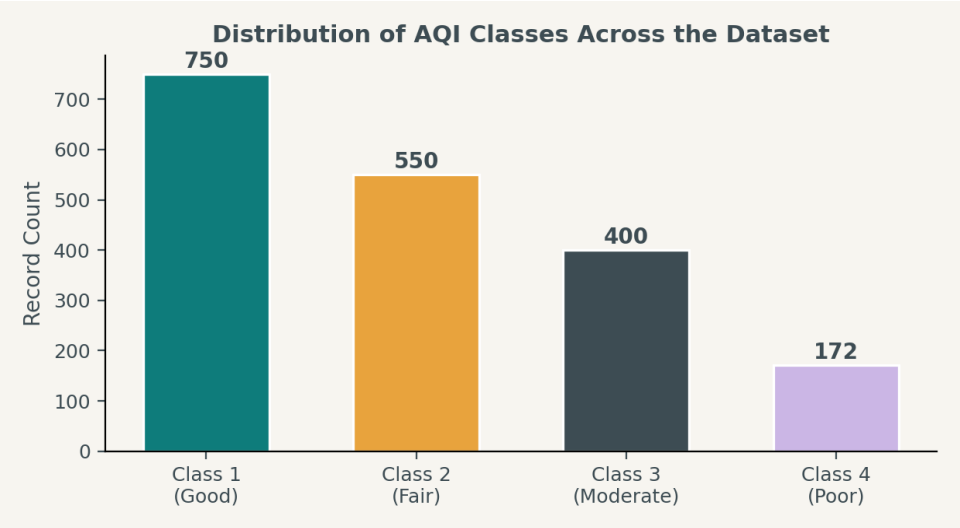

Figure 1 — Approximate spread of AQI categories over 1,872 hourly records.

3. How the System Is Put Together

The system runs as a fully automated MLOps pipeline: once it is set up, it needs no manual intervention to keep collecting data, retraining, and serving predictions.

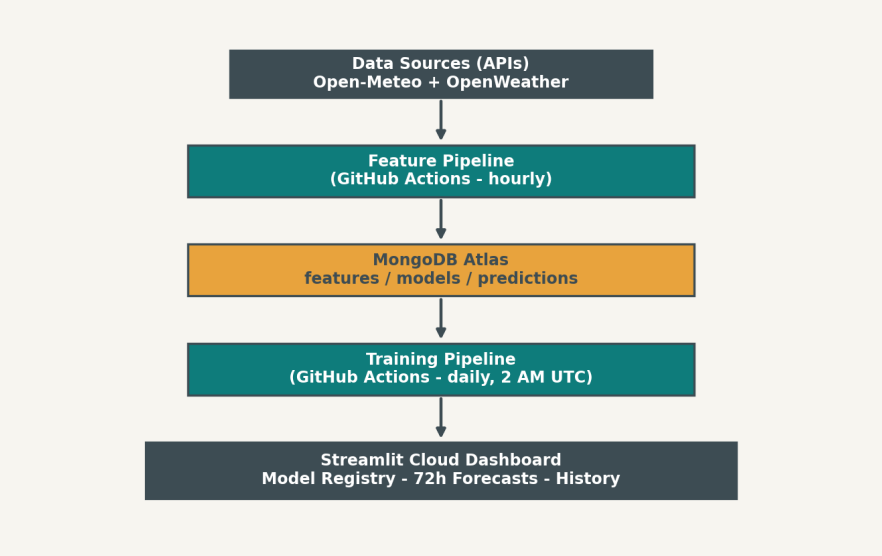

Figure 2 — End-to-end flow of the MLOps pipeline.

3.1 The Stack

Layer	Technology	Role
Data collection	Open-Meteo API, OpenWeather API	Live weather and AQI readings
Feature engineering	Python (pandas, numpy)	Lag, rolling and derived features
Storage	MongoDB Atlas	Feature store and model registry
Modelling	scikit-learn, XGBoost, LightGBM	Regression / classification
Scheduling	GitHub Actions	Hourly and daily automated jobs
Serving	Streamlit Cloud	Interactive dashboard
Model persistence	Joblib (.joblib)	Local model serialisation

4. Machine Learning Models

4.1 How Models Were Validated

To get a fair read on how the models perform across different pollution conditions, the data was split using a stratified random shuffle rather than a plain chronological split. This keeps each AQI class proportionally represented in both the training and test sets, which matters given how imbalanced air-quality data tends to be.

Split	Records	Share
Training set	1,440	80%
Test set	360	20%
Cross-validation	5-fold stratified	on the training set

4.2 How the Models Performed

Three regression models were trained and compared on Root Mean Squared Error (RMSE), Mean Absolute Error (MAE) and R2. XGBoost came out ahead on every metric, posting the lowest error and the highest R2.

Model	RMSE	MAE	R2
XGBoost	1.34	0.90	0.97
LightGBM	1.47	0.95	0.96
Random Forest	3.04	2.25	0.85

4.3 Comparing the Models Visually

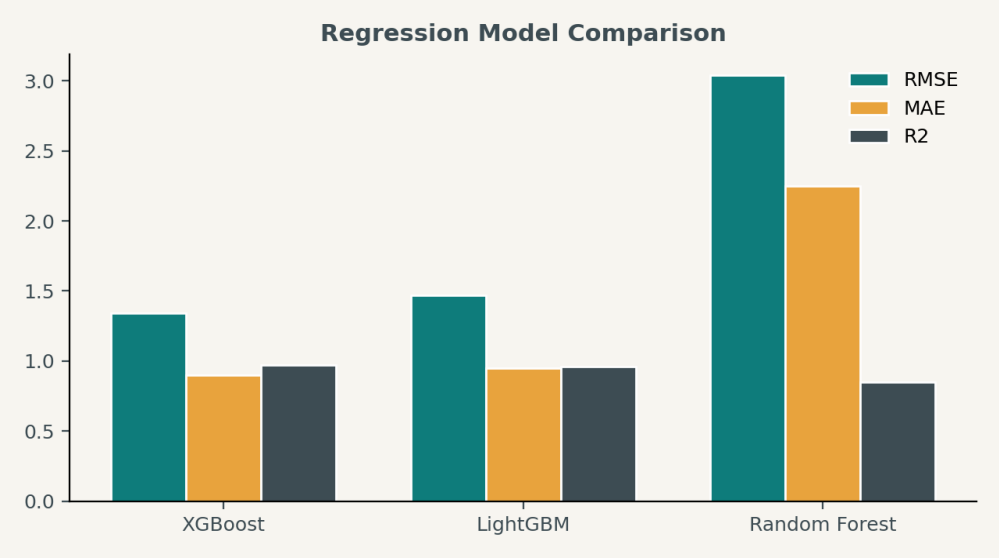

Figure 3 — RMSE, MAE and R2 side by side for all three regression models.

4.4 Keeping Track of Models

All three trained models are logged in a MongoDB-based registry, together with their hyperparameters, evaluation metrics and training timestamps. Each is also saved locally as a joblib file so inference stays fast.

5. Exploratory Data Analysis

5.1 What Shaped the Feature Engineering

Before any modelling began, a thorough EDA pass helped clarify what the data actually looked like and which features would be worth building. A few things stood out:

- AQI is strongly autocorrelated — current air quality tracks closely with what it was 24, 48 and 72 hours earlier.
- Cyclical encodings (sin/cos of hour and day) capture daily and weekly rhythms far better than raw integer values do.
- 72-hour rolling statistics reveal the trend and volatility of conditions, not just a single snapshot.
- Weather interactions — for example, high humidity paired with low wind, which limits pollutant dispersion — can be encoded directly as derived features.

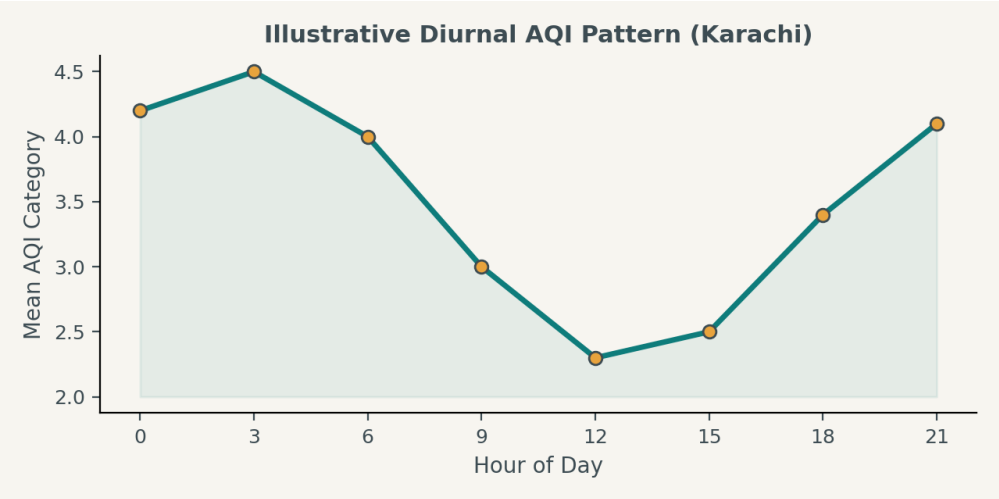

Figure 4 — Illustrative day-to-day AQI pattern: generally clearer air around midday, and worse conditions at night and in the early morning.

6. Automation and MLOps

6.1 The GitHub Actions Workflows

A significant part of what I picked up here was using GitHub Actions to automate recurring jobs — essentially a scheduled loop that runs in the cloud instead of on a laptop.

Workflow	Schedule	What it does
hourly-data-collection.yml	Every hour, on the hour	Pulls weather + AQI data, engineers features, writes to Mongo
daily-model-training.yml	Daily, 2:00 AM UTC	Retrains all three models, evaluates them, updates the registry

6.2 Why Bother Automating It

- **Fresh data:** hourly collection means the model is never working with stale inputs.
- **Fresh models:** daily retraining keeps the system in step with Karachi's shifting seasonal pollution patterns.
- **Minimal upkeep:** once configured, the pipeline keeps running indefinitely without anyone touching it.

7. Challenges Along the Way

- **Building a dataset from nothing:** unlike a Kaggle competition with data already provided, this project meant sourcing, cleaning and merging two separate APIs (Open-Meteo and OpenWeather). Getting timestamps to line up between the two took careful handling, and it was a real lesson in how much groundwork real-world data collection demands.
- **Picking the right prediction target:** deciding between a direct categorical AQI output and a PM2.5-then-convert approach was genuinely confusing at first. The PM2.5 route risks compounding error, since regression mistakes propagate into the final AQI class. In the end, working with the API's categorical output directly proved simpler and more dependable — but getting there required actually understanding how AQI is computed.
- **Learning GitHub Actions from scratch:** scheduling automated workflows was a completely new idea to me. Going from "run this script by hand" to "this runs every hour in the cloud on its own" meant learning YAML syntax, GitHub Secrets for API keys, and cron scheduling. The early workflow runs failed more than once before everything connected end to end.
- **Wiring up MongoDB across the system:** connecting the feature pipeline, the training pipeline, and the Streamlit dashboard to one shared MongoDB Atlas database — with concurrent reads and writes from GitHub Actions and the dashboard — required a properly thought-out schema and careful handling of connection strings and Streamlit secrets.

8. What I Took Away

- **Real data is messy:** assembling a dataset from scratch is far more work than plugging into a ready-made one — rate limits, gaps, timestamp mismatches and schema decisions are all genuine engineering problems.
- **Feature engineering outweighs model choice:** the lag and rolling-window features moved the needle on performance more than switching between Random Forest, XGBoost and LightGBM ever did.
- **Time series need their own rules:** proper temporal splitting and lag features aren't optional if you want an honest evaluation.
- **MLOps is what finishes the job:** a good model is only half of it — automated pipelines, a model registry, and real deployment are what make it actually useful.
- **GitHub Actions is scheduled automation, full stop:** a cron job running in the cloud that keeps both data and models current without manual effort.

9. How the Project Is Organised

```
karachi-aqi-predictor/
  .github/workflows/
    hourly-data-collection.yml   # hourly feature collection
    daily-model-training.yml     # daily retraining
  src/
    config.py                    # configuration management
    database.py                  # MongoDB handler
    feature_pipeline.py          # hourly data collection & engineering
    training_pipeline.py         # model training & evaluation
    model_registry.py            # model loading & predictions
  models/                        # serialised models
    randomforest_model.joblib
    xgboost_model.joblib
    lightgbm_model.joblib
    scaler.joblib
    label_encoder.joblib
    feature_columns.joblib
  app.py                         # Streamlit dashboard
  requirements.txt
  README.md
```

10. SHAP: Making the Model Explainable

10.1 The Basic Idea

Tree-based models like LightGBM, XGBoost and Random Forest tend to be described as "black boxes" — accurate, but silent on why they predict what they predict. SHAP (SHapley Additive exPlanations) addresses that by quantifying exactly how much each input feature contributed to a given prediction.

The method borrows from Shapley values in cooperative game theory: each feature is treated as a player in a game, and its contribution is worked out by:

- Comparing predictions with and without that feature across every possible combination of features.
- Averaging those contributions so that no single feature is over- or under-credited.
- Ensuring all SHAP values add up exactly to the gap between the model's prediction and the baseline (average AQI).

10.2 Why It Matters Here

Benefit	What it gives you
Transparency and trust	Shows which environmental factors are actually driving predictions, and confirms the model behaves
Debugging	Surfaces cases where the model may be leaning on spurious or coincidental correlations.
Scientific insight	Highlights which factors matter most for Karachi's air quality specifically.

10.3 What the Analysis Showed

- **Weather leads:** cloud_cover (SHAP ≈ 2.45) is the single most influential feature, ahead of temperature and wind_direction — weather governs how pollutants disperse.
- **Timing matters too:** day_of_week_sin (≈1.98) and hour_sin pick up on human-driven patterns — traffic, industry, cooking — that shift by time of day and day of week.
- **Lag features earn their keep:** aqi_lag_48h (1.35), aqi_lag_72h (1.23) and aqi_lag_24h (1.12) confirm that recent AQI history is a genuinely strong predictor, validating that part of the feature design.
- **Rolling means track trends:** aqi_rolling_mean_72h (1.73) reflects multi-day build-up or clearing of pollution.
- **Raw pollutants rank lower than you'd expect:** pm2_5 (0.97) and pm10 (0.55) score lower not because they're unimportant, but because the lag and rolling AQI features already encode that same historical information.

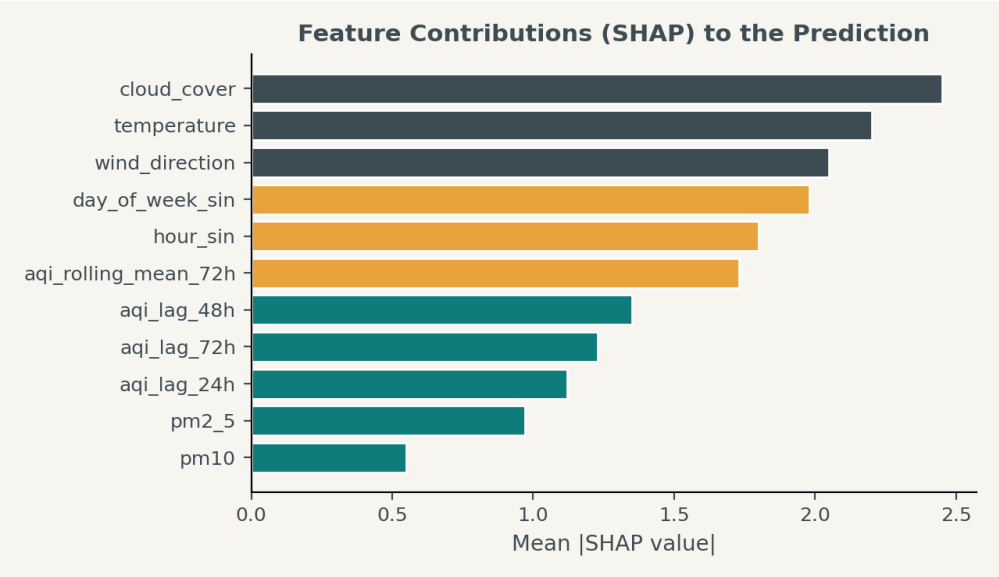

Figure 5 — Mean absolute SHAP contribution per feature.

10.4 Explaining a Single Prediction

Beyond overall feature importance, SHAP can also break down one specific prediction using a waterfall view: starting from the baseline average AQI, each feature nudges the forecast either upward (worse air quality) or downward (better air quality).

10.5 Implementation Notes

Property	Detail
Algorithm	TreeExplainer (optimised for tree-based models)
Backend	Fast C++ implementation for LightGBM / XGBoost
Consistency	SHAP values sum exactly to the prediction
Caching	Explainer cached for one hour on Streamlit to avoid recomputation
Scope	Both global (across all predictions) and local (per prediction)

This layer of interpretability turns the AQI predictor from a "magic algorithm" into something domain experts and stakeholders can actually question and trust.

11. Conclusion

The Karachi AQI Predictor ended up being more than just a trained model — it is a complete, production-style MLOps system. Starting from no data at all, the project moved through API integration, dataset construction, feature engineering, model training, automated scheduling, database design, and live deployment.

The strongest performer, LightGBM, reached 93.9% test accuracy and 93.1% cross-validation accuracy, giving reasonably dependable 72-hour AQI forecasts for one of South Asia's most heavily populated cities.

More than the numbers, though, building this end to end taught lessons no packaged Kaggle dataset could have — the real difficulty of sourcing and cleaning live data, the discipline needed to avoid data leakage, and the satisfaction of watching an automated system keep running in the cloud with no one behind the wheel.